2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT) | 979-8-3503-7024-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICCCNT61001.2024.10725955

Weighted Average Ensembled Probability Pipeline of Pre-trained CNN Models for Hand Gesture Classification

Rahul Gunti
Department of Computer Science and Engineering
Vardhaman College of Engineering
Hyderabad, India
rahulgunti08@gmail.com

Rohan Reddy B
Department of Computer Science and Engineering
Amrita School of Computing
Amrita Vishwa Vidyapeetham Chennai-601103, India
rohanbadugula@gmail.com

Gunti Swathi
Department of Computer Science and Engineering
Amrita School of Computing
Amrita Vishwa Vidyapeetham Chennai-601103, India
swathigunti4@gmail.com

Sharath Kumar Reddy G
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
guvvalasharath@gmail.com

***Abstract*—The automatic detection of sign language from hand gesture images is fundamental for effective human-computer interaction, especially for individuals with hearing and speech disorders. Achieving accurate detection and classification of sign language gestures is paramount. However, accurately recognizing these gestures frequently presents challenges due to dull backgrounds, variations in gesture orientation, and fluctuations in lighting conditions. Previous approaches tried to overcome these challenges but still the model performance lacks recognition accuracy and visual quality. To address this need for accuracy, we proposed a Weighted Average Ensembled Probability Pipeline composed of EfficientNetB7, ResNet-50, and Xception models. Leveraging the Extensive Hand Gesture Recognition Database (HGRD), comprising 20,000 images, our individual models achieved notable accuracies: 96.498% for EfficientNetB7, 93.2% for ResNet-50, and 90.175% for Xception. Through ensemble learning, we integrate these models to attain an impressive overall accuracy of 98.95%. This research contributes to robust gesture recognition system and underscores the significance of ensemble techniques in enhancing performance for Gesture recognition systems.**

***Index Terms*—Ensemble Model, EfficientNetB7, ResNet-50, Xception, pre-tained CNN, Hand Gesture Recognition**

I. Introduction

With a growing global population of deaf individuals, there is an increasing focus on advancements in hand gesture recognition. The World Health Organization reports approximately 400 million people worldwide who are either deaf or hard of hearing, representing around 20% of the total global population [1]. Sign language or gestures acts as the sole primary method of communicating communication for individuals with hearing and speaking impairments. It operates as a as a visual-gestural language that facilitates the exchange of ideas and thoughts among them. By combining hand shapes, orientation, and movements, sign language effectively communicates the speaker's thoughts through manual expression [2]. However, the complexity of sign language poses a challenge for many individuals, making it as a hurdle to the deaf community to communicate with the general populace , and vice versa.

To address this challenge, hand gesture recognition systems have emerged, employing deep learning to automate the process of gesture recognition by machines [3]. These systems not only enable communication but also find applications in facilitating human-robot interaction (HRI), sign language interpretation, controlling computer games, enhancing virtual reality experiences, and creating assistive environments. Furthermore, they play a significant role in supporting speech-impaired individuals by improving access to education, employment opportunities, and societal rights.[4] Despite this, developing a robust gesture recognition system is challenging due to the need for it to operate effectively in diverse environments with varying backgrounds and lighting conditions, while also considering different gesture positions, orientations, and occlusions. Conventional methods rely on digital image processing and various machine learning and deep learning techniques to comprehend and interpret sign language. However, these systems encounter significant complexities from the external environment during development. In this research, we present a weighted average ensemble probability pipeline comprising EfficientNetB7 [5], ResNet-50 [6], and Xception [7]. This approach harnesses the individual strengths of these models while mitigating their respective disadvantages.

II. Related Works

With the continuous evolution of technology, gesture recognition systems garnered attention in everyday applications aiding deaf individuals. There were numerous machine learning models employed for sign language recognition. Starner

et al.(2002) [8] pioneered a real-time system using Hidden Markov Models for recognizing English Sign Language, focusing on overall hand gestures rather than explicitly modeling fingers. In another study by Naresh et al.(2017) [9], discrete wavelet transform was utilized to extract features from hand patch images, followed by dimensionality reduction using linear discriminant analysis. Classification was then performed using both LDA and SVM algorithms. Dutta et al.(2017) [10] utilized MATLAB to classify single and double-handed Indian sign language. They employed dimensionality reduction using PCA and utilized both K-Nearest Neighbors and Backpropagation techniques.

Rahman et al.(2023) [1] detailed a real-time sign language recognition system that had over 3,000 movement samples. Machine Learning algorithms like KNN, SVM, and ANN were used to conduct the classification. Nidhi et al.(2023) [3] utilized OpenCV in addition with ML models like Random Forest, SVM, Decision Tree and KNN classifiers to capture alphabet and digit signs from a dataset. They have also employed preprocessing steps, including resizing and binary conversion, recognition was facilitated through a user-friendly keyboard interface supporting text and audio output in both English and Hindi. Dawod et al.(2019) [2] devised a framework utilizing machine learning techniques such as Hidden Conditional Random Field (HCRF) and Random Decision Forest (RDF) that automatically emphasizes dynamic isolated signs. The framework addressed challenges like low lighting conditions and cluttered backgrounds. Hemachandran et al.(2022) [11] utilized background subtraction and edge detection preprocessing techniques alongside ORB and SIFT feature extraction methods which helped the multiple machine learning models like Decision Tree, SVM, Logistic Regression and MLP in classification.

In recent years, deep learning models have emerged as a leading technology for classification, overcoming challenges faced by traditional machine learning algorithms. Mallika et al.(2023) [12] utilized a CNN to classify hand signs in sign language detection. They have utilzed a optimal combination of some preprocessing techniques that has enabled accurate recognition of sign language gestures. Similarly, Nagi et al.(2011) [13] developed a real-time hand gesture-based interface for human-robot interaction on mobile robots. They employed a deep neural network, combining convolution and max-pooling , to classify 6 gesture classes.

Rukumani et al.(2023) [14] utilized the VGG16 model, with diverse images of each sign to enhance robustness across varied scenarios. The system accurately recognized all ASL alphabets and improved communication between deaf and hearing individuals. Bhadra et al.(2021) [4] achieved satisfactory performance in classifying both static and dynamic gestures using a deep CNN for automatic sign language detection from hand gesture images. Marc et al (2022) [15] employed a comparative analysis of ResNet, a Pruned VGG network, and 1D-CNN for hand landmark coordinates classification. They achieved superior performance by employing unprocessed raw images alongside a Pruned VGG network.

III. Dataset Description

Hand gestures are fundamental to human-computer interaction, particularly with the rapid advancements in augmented reality and virtual reality technologies. In the current research, we leverage the Hand Gesture Recognition Database (HGRD) [16], a publicly available dataset comprising 10,000 images captured using the Leap motion sensor. The dataset encompasses ten distinct hand gestures widely employed for interfacing with technological environments: palm, L, fist, fist moved, thumb, index, OK, palm moved, C, and down. This data is collected from ten subjects (five men and five women), each gesture class consists of 200 images per subject, resulting in a total of 2000 images per image gesture.

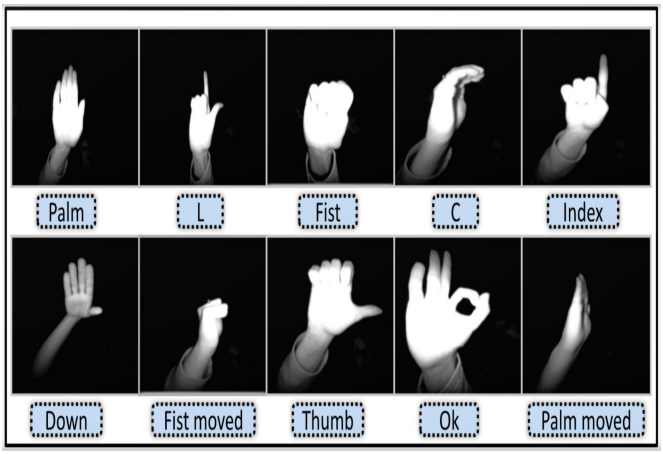


Fig. 1: Representation of Hand Gestures in the Comprehensive Dataset

Visual representations of each gesture class are depicted in Figure 1. Noteworthy pre-processing and augmentation techniques have been applied to ensure data integrity and enhance the dataset's utility for analysis. We have employed a standard split of 70:10:20 for train set, validation set and test set respectively.

IV. Methodology

In our study, we propose a Weighted Average Ensemble Probability Pipeline [17] comprising EfficientNetB7, ResNet50, and Xception models to classify the ten hand gesture classes as shown in Figure 2. The ensemble approach was imperative due to the dataset's extensive size and the diversity of class types. EfficientNetB7, ResNet50, and Xception are chosen for their diverse network architectures and underlying feature extraction strategies. Each of these three pretrained models provides unique attention to image features that in-turn brodens the perspective on the provided samples and enhances the system's overall classification. By aggregating predictions from these independent pre-trained models, we obtain a terminal model that harnesses the strengths of individual models while mitigating potential weaknesses, thereby attaining the robustness in classification accuracy. The combination of these distinct architectures holds promise for

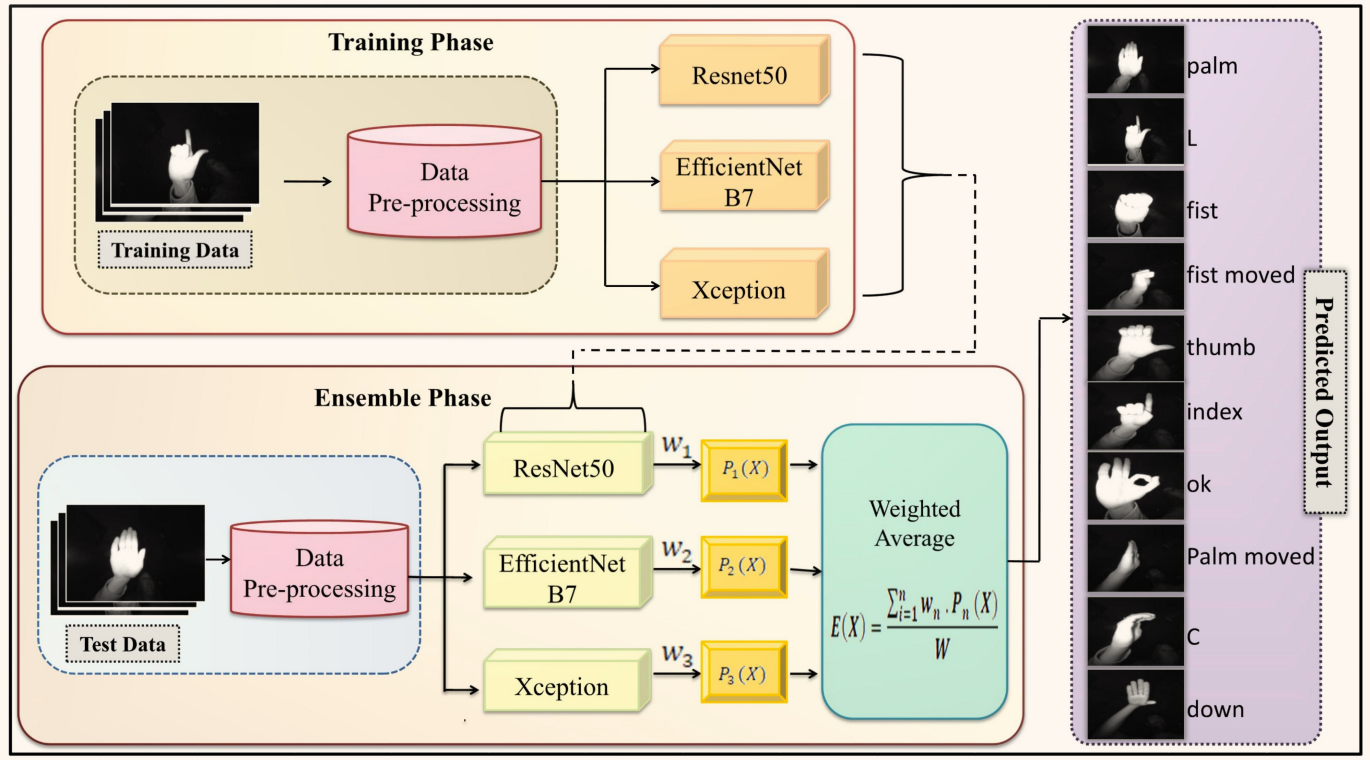


Fig. 2: Weighted Average Ensembled Probability Pipeline

advancing gesture recognition systems, particularly in complex real-world environments.

A. *EfficientNetB7*

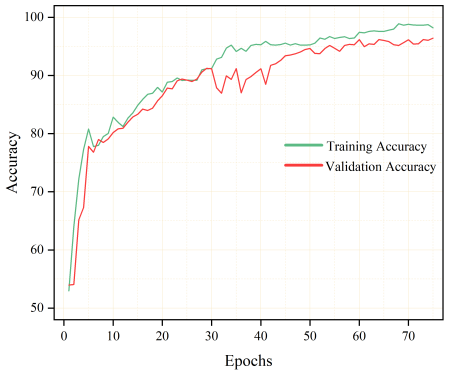
The EfficientNetB7 was proposed by google brain, the model stands as a pinnacle achievement in neural architecture design. The architecture is crafted through a comprehensive neural architecture search algorithm that provides a paradigm shift towards achieving state-of-the-art accuracy. By judiciously scaling dimensions such as width, depth, and resolution, the EfficientNet models achieve a delicate equilibrium between model size and performance. Notably, when the computational demands increase linearly with respect to model's size efficiency becomes a paramount concern. In the series EfficientNetB7 is the largest model containing 800 layers and around 66 million trainable parameters. At its core, the EfficientNetB7 architecture inherits crucial components from MnasNet [18] like the mobile inverted bottleneck MBConv blocks complemented by SE (Squeeze-Excitation) Blocks. These components, coupled with depth-wise convolution filters and compound scaling, provides a remarkable accuracy within constrained computational budgets. The network is Organized in a hierarchical manner to progressively extract intricate features from input images across multiple layers. This utilization of computational scaling strategies provides a optimal balance between computational efficiency and precision.

B. *ResNet-50*

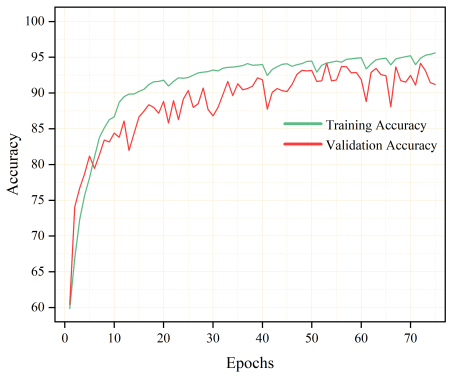
ResNet-50 is prominent architecture in the Residual Network series that has introduced residual connections to address the vanishing gradient problem in deep architectures. ResNet50 entails 50 layers and 25.6 million trainable parameters, primarily structured with residual blocks that contain multiple convolutional layers intertwined with shortcut connections among them. This enables a direct gradient flow during backpropagation. This Architecture eliminates the challenges of training deep architectures and enables the model to learn intricate representations efficiently. Utilizing 3x3 convolutions within residual blocks strikes a balance between receptive field size and computational efficiency, enhancing the model's learning capabilities. Moreover, integrated max-pooling layers enable spatial down-sampling, further enhancing the model's effectiveness. ReLU activation functions in hidden layers promote nonlinear transformations crucial for capturing complex patterns. Batch normalization is employed to stabilize training by normalizing layer activations, thus accelerating convergence and enhancing robustness.

C. *Xception*

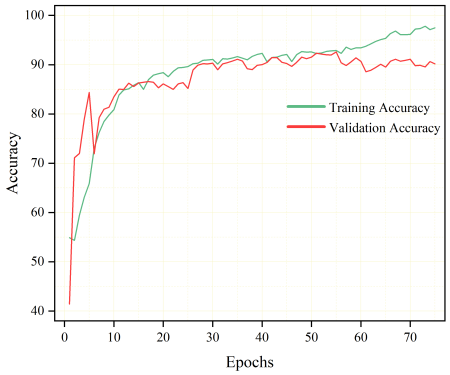
The Xception architecture, derived from "Extreme Inception," integrates elements of the Inception architecture while incorporating depth-wise separable convolutions. Departing from conventional approaches where 1x1 convolutions are first utilized to compress the input before filtering across depth spaces, Xception reverses this process. Initially, filters



3.a: EfficientNetB7

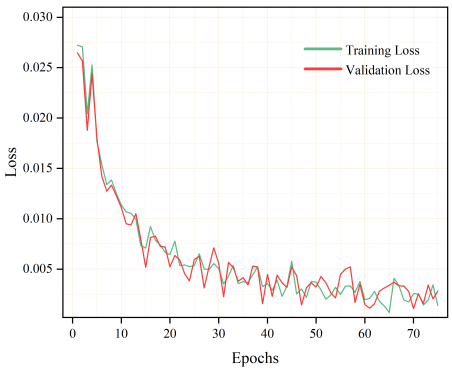


3.b: ResNet-50

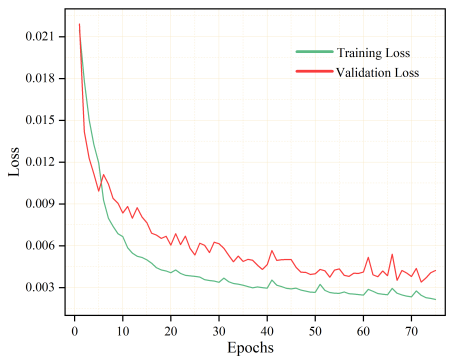


3.c: Xception

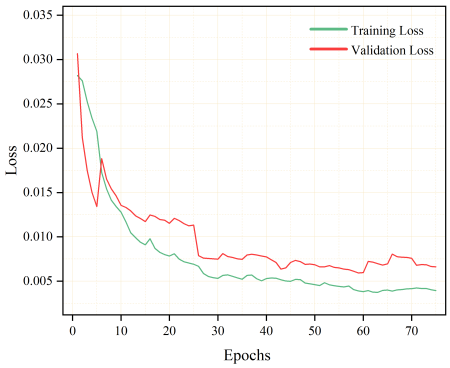
Fig. 3: Plots of Training and validation accuracies



4.a: EfficientNetB7



4.b: ResNet-50



4.c: Xception

Fig. 4: Plots of Training and validation losses

are applied to each depth map, followed by compression via 1x1 convolutions. Unlike Inception, Xception does not introduce non-linearity, distinguishing it in architectural design. Comprising two phases, Xception consists of depth-wise convolutions (DWC) and point-wise convolutions (PWC). In DWC, each input undergoes convolution independently with its corresponding filter set. Subsequently, PWC employs 1x1 convolutions to amalgamate the output of depth-wise convolutions. This approach notably diminishes computational demands, rendering Xception a lightweight model. The architecture encompasses 22.9 million trainable parameters and 36 convolutional layers functioning as feature extractors. The network is structured into 14 modules, each encompassing linear residual connections. These connections facilitate information flow across modules, enhancing the network's representational capabilities.

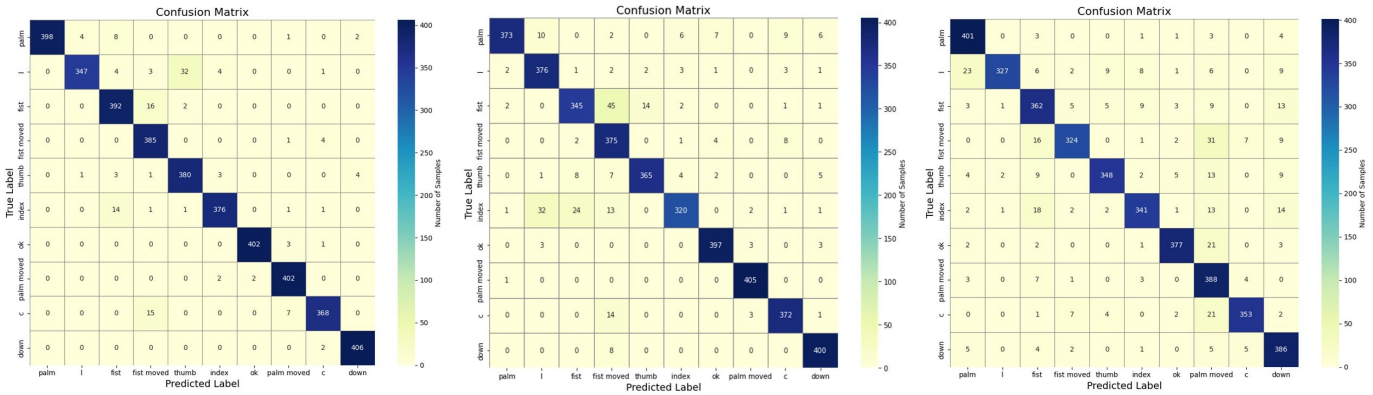
D. Weighted Average Ensembled Probability Pipeline

The process of ensembling involves amalgamating the predictive capabilities of individual models to yield comprehensive predictions. By leveraging the collective strengths of constituent models, ensembling endeavours to mitigate potential drawbacks inherent in any single model. In this study, we introduce a structured weighted average ensembled probability pipeline. This pipeline comprises two distinct phases: the training phase and the Ensembling phase. During the training phase, the EfficientNetB7, ResNet50, and Xception models are trained on the pre-processed training data. Subsequently, in the Ensembling phase, the trained models are subjected to test data samples to generate prediction probability vectors. These prediction vectors encapsulate the models classification result containing the probability of each class. To derive a final prediction vector, a weighted average aggregation technique is employed, to combine the prediction vectors from each model given by,

$$E(\mathbb{X}) = \frac{w_1 \cdot V_1(\mathbb{X}) + w_2 \cdot V_2(\mathbb{X}) + w_3 \cdot V_3(\mathbb{X})}{W} \quad (1)$$

Here in Equation 1 $\mathbb{X}$ is the test sample, and $E(\mathbb{X})$ is the final predicted probability vector of the Ensemble model corresponding to $\mathbb{X}$. Additionally, $V_1(\mathbb{X})$, $V_2(\mathbb{X})$, and $V_3(\mathbb{X})$ represent the probability vectors of the EfficientNetB7, ResNet50, and Xception models, respectively. The weights assigned to each model are denoted by w_1, w_2, and w_3, with W representing their summation, equated to 1 to ensure proportionality.

Assigning suitable weights to individual model is crucial to ensure that each model's contribution is duly considered in the ensemble. Considering this we have leveraged the Grid Search



5.a: EfficientNetB7 5.b: ResNet-50 5.c: Xception

Fig. 5: Confusion matrices of the three classifiers

technique to best fit weights. This method systematically explores all possible combinations of weights to determine the optimal configuration that maximizes the ensemble model's accuracy.

$$W^* = argmax_{W \in \Omega} Acc(E(\mathbb{X}_{val}; W)) \tag{2}$$

The Equation 2 illustrates the mathematical representation of the Grid Search. It within the framework of an optimization problem, wherein the objective is to identify the optimal weight vector (W^*). This vector is derived from the feasible set (Ω),

$$\Omega = \left\{ W \in \mathbb{R}^3 : w_1 + w_2 + w_3 = 1 \right\} \tag{3}$$

In Equation 3,Ω encompasses all combinations of weights that adhere to the constraint of their summation equating to 1. By evaluating the performance of the ensemble model with all possible weight configurations, the Grid Search method computes of the weight vector that yields the highest achievable accuracy.

V. Experimental Analysis

The experimental setup involved the simulation of all models on a NVIDIA TESLA P100 GPU. The categorical cross-entropy (CCE) [19] loss function mentioned in Equation 4 is utilized for optimization, facilitating the minimization of dissimilarities between true class labels and predicted classes.

$$CCE(t, \hat{t}) = -\sum_{i=0}^{n-1} t_i \cdot \log(\hat{t_i}) \tag{4}$$

Here, t represents a one-hot encoded vector of true classes, $\hat{t}$ signifies the predicted probability distribution, and n denotes the number of classes. Additionally, the Adam optimizer [20] was chosen for updating model parameters during training. The optimization process is governed by the equation:

$$P_{t+1} = P_t - \theta \times \frac{\hat{x_t}}{\sqrt{\hat{y_t} + \epsilon}} \tag{5}$$

Here in Equation 5, P_t are the parameters at time step t, θ is the learning rate which is set to 0.001, $\hat{x_t}$ and $\hat{y_t}$ signify bias-corrected estimates of the first and second moments of gradients, respectively. To avoid division by zero, a small constant ϵ is added.Additionally, if the loss metric remains stagnant for more than 5 epochs, a reduction factor of 0.2 is applied. Furthermore, the SoftMax Activation [21] function was implemented in the final classification layer of all models to yield a probability distribution. The Experimental setup

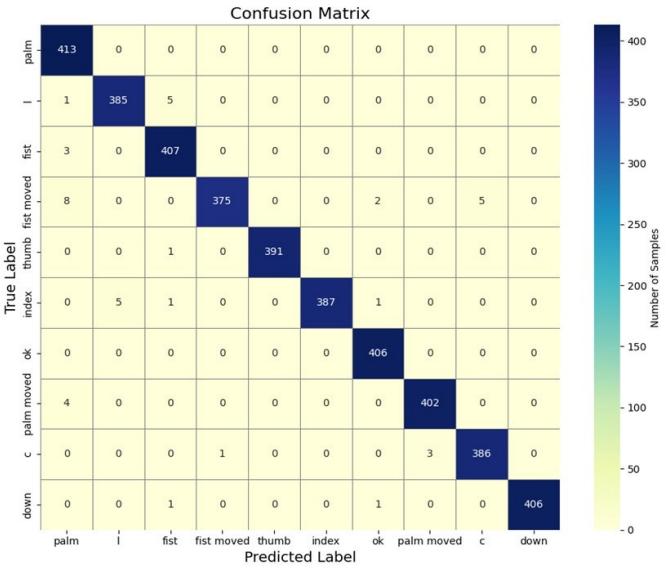


Fig. 6: Confusion Matrix of Final Ensemble Model

and Hyperparameters are chosen after the rigorous trying of all different possibilities. All the three models underwent a training of 75 epochs, the batch size was set to 32 and an "EarlyStopping" mechanism was set up with a patience parameter of three. To save the best model weights, model checkpointing was implemented during the training process. The training and validation plots of all the three models are depicted in Figure 3 and Figure 4.

VI. Results and Discussion

The implementation of the Grid Search technique resulted in optimal weights for the Ensemble model as (0.5, 0.3, 0.2) for EfficientNetB7, ResNet50, and Xception, respectively. To assess the model's efficacy, we utilized various performance metrics including Accuracy score, Precision, Recall, and F1-Score [22]. The comprehensive results of individual models and the Ensemble model are given in Table I. Additionally, the confusion matrices depicting the performance of three models are illustrated in Figure 5 and Final Ensemble model is provided in Figure 6.

TABLE I: Performance metric results of all the models

Models	Accuracy	Precision	Recall	F1-Score
EfficientNet B7	96.498	96.489	96.371	96.372
Resnet50	93.200	93.458	93.207	93.159
Xception	90.175	90.971	90.010	90.257
Ensemble Model	**98.950**	**98.980**	**98.935**	**98.949**

The analysis reveals that EfficientNetB7 achieved the highest accuracy of 96.498%, followed by ResNet50 with 93.2%, and Xception with 90.175% accuracy. Notably, our goal to maximize accuracy was successfully realized through the Weighted Average Ensembled Probability Pipeline, yielding an accuracy of 98.950%. Additionally, the confusion matrices of all the models, providing insights into their performance across different classes. The Ensemble model's confusion matrix underscores its ability to effectively address specific class challenges by leveraging the collective strengths of individual models.

VII. Conclusion

To recapitulate, this research introduced a hand gesture recognition systems through a Weighted Average Ensembled Probability Pipeline comprising EfficientNetB7, ResNet50, and Xception models. Our ensemble model achieves an outstanding accuracy of 98.950%, surpassing previous works and demonstrating its effectiveness. Looking ahead, integrating object detection techniques with a classification pipeline presents a promising avenue for further improvement, particularly in extending the system's applicability to video streams. This advancement heralds a significant stride towards more accurate and versatile human-computer interaction in virtual reality environments.

References

[1] M. Z. U. Rahman, M. A. Akbar, M. Usman, M. H. Tahir, Shahzaib, M. T. Riaz, and M. A. Khan, "A real-time system for classification of pakistan sign language using machine learning," pp. 1–5, 2023.

[2] A. Y. Dawod and N. Chakpitak, "Novel technique for isolated sign language based on fingerspelling recognition," pp. 1–8, 2019.

[3] N. Chandarana, S. Manjucha, P. Chogale, N. Chhajed, M. G. Tolani, and M. R. M. Edinburgh, "Indian sign language recognition with conversion to bilingual text and audio," pp. 1–7, 2023.

[4] R. Bhadra and S. Kar, "Sign language detection from hand gesture images using deep multi-layered convolution neural network," pp. 196–200, 2021.

[5] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," 2020.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 2015.

[7] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," 2017.

[8] T. Starner and A. Pentland, "Real-time american sign language recognition from video using hidden markov models," *Proceedings of the IEEE International Conference on Computer Vision*, pp. 265 – 270, 12 1995.

[9] N. Kumar, "Sign language recognition for hearing impaired people based on hands symbols classification," pp. 244–249, 2017.

[10] K. K. Dutta and S. A. S. Bellary, "Machine learning techniques for indian sign language recognition," pp. 333–336, 2017.

[11] B. Hemachandran, C. Pavan Rakesh Reddy, and D. Harsha Vardhan Reddy, "Comparative study of classification algorithms in sign language recognition," pp. 1–5, 2022.

[12] S. Sugantha Mallika, M. Priyadharsini, S. Samritha, C. Sowmiya, and B. Nikitha, "Hand gesture recognition using convolutional neural networks," pp. 249–255, 2023.

[13] J. Nagi, F. Ducatelle, G. A. Di Caro, D. Cireşan, U. Meier, A. Giusti, F. Nagi, J. Schmidhuber, and L. M. Gambardella, "Max-pooling convolutional neural networks for vision-based hand gesture recognition," pp. 342–347, 2011.

[14] R. C, V. P, S. S. C, A. G, C. N, and R. B. C, "Deep learning based smart american sign language detection in alphabets," vol. 6, pp. 2333–2338, 2023.

[15] M. Marais, D. Brown, J. Connan, and A. Boby, "An evaluation of hand-based algorithms for sign language recognition," pp. 1–6, 2022.

[16] A. Kapitanov, K. Kvanchiani, A. Nagaev, R. Kraynov, and A. Makhliarchuk, "Hagrid - hand gesture recognition image dataset," 2024.

[17] G. Swathi, R. P. Kumar, B. M. G, and E. R, "Ensemble integration of deep learning models for gender-based speech emotion recognition," pp. 1–7, 2023.

[18] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, and Q. V. Le, "Mnasnet: Platform-aware neural architecture search for mobile," 2019.

[19] Z. Zhang and M. R. Sabuncu, "Generalized cross entropy loss for training deep neural networks with noisy labels," 2018.

[20] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017.

[21] C. M. Bishop, "Pattern recognition and machine learning," *URL http://proceedings. mlr. press/v15/glorot11a. html*, 2006.

[22] D. M. W. Powers, "Evaluation: from precision, recall and f-measure to roc, informedness, markedness and correlation," 2020.